Certification Project-Insurance Domain Project

Project Title: InsureMe

Submitted by - Kalimuthu A

Date: 10-04-2026

Project Overview:

The InsureMe project is a cloud-based insurance management application designed to demonstrate a complete DevOps CI/CD pipeline using modern tools and AWS services.

This application is developed using Python (Flask) and allows users to manage insurance policies through REST APIs such as creating, viewing, updating, and deleting policies.

To ensure scalability, reliability, and automation, the application is containerized using Docker and deployed on AWS ECS (Fargate). The Docker images are stored in Amazon ECR, and the entire build and deployment process is automated using AWS CodePipeline and CodeBuild.

Whenever a developer pushes code to GitHub, the pipeline is automatically triggered. The application is built, containerized, pushed to ECR, and deployed to ECS without any manual intervention.

An Application Load Balancer (ALB) is used to expose the application to users, ensuring efficient traffic distribution and high availability.

InsureMe architecture Diagram on AWS cloud:

Technologies Used :

Python (Flask)

Docker

AWS Code Pipeline

AWS Code Build

Amazon ECR

Amazon ECS (Fargate)

Application Load Balancer

GitHub

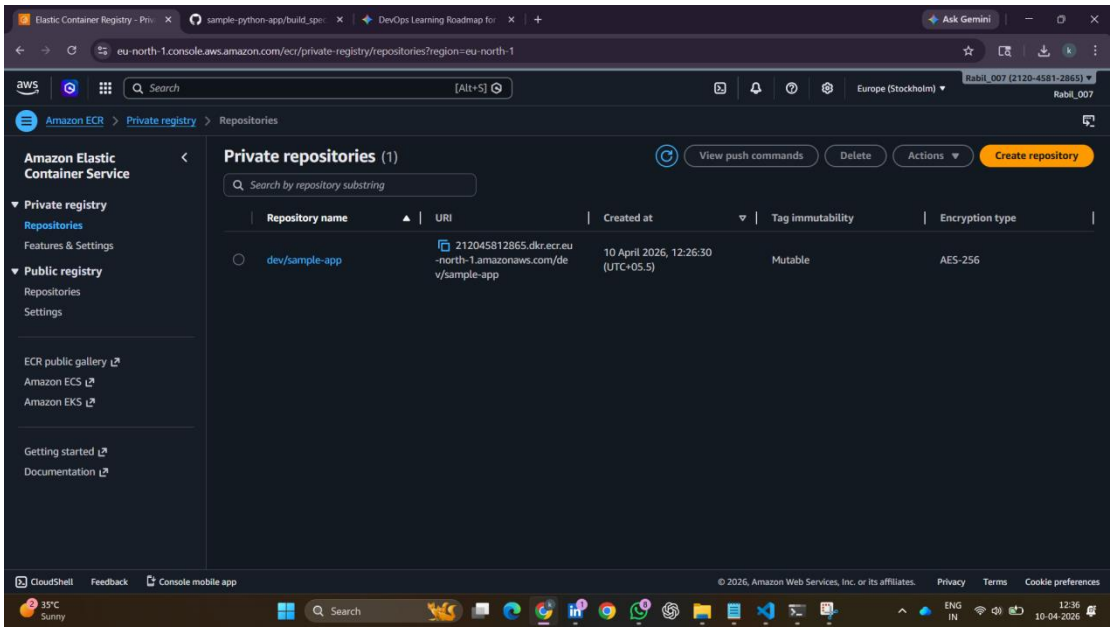
Steps:

1.Create ECR (Elastic Container Registry)

Go to AWS Console → ECR

Create repository : project/insure-me

Save repository URI



2. Create requirements.txt

flask
flask_sqlalchemy
gunicorn

2. Create Application Code

Developed Flask application (app.py)

Created APIs:

Create Policy

View Policy

Update Policy

Delete Policy

Added frontend (index.html)

4. Create Docker file

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["gunicorn", "-b", "0.0.0.0:5000", "app:app"]
```

5. Create buildspec.yml :

Used to automate build and push process

Key steps:

Login to ECR

Build Docker image

Push image to ECR

Create imagedefinitions.json

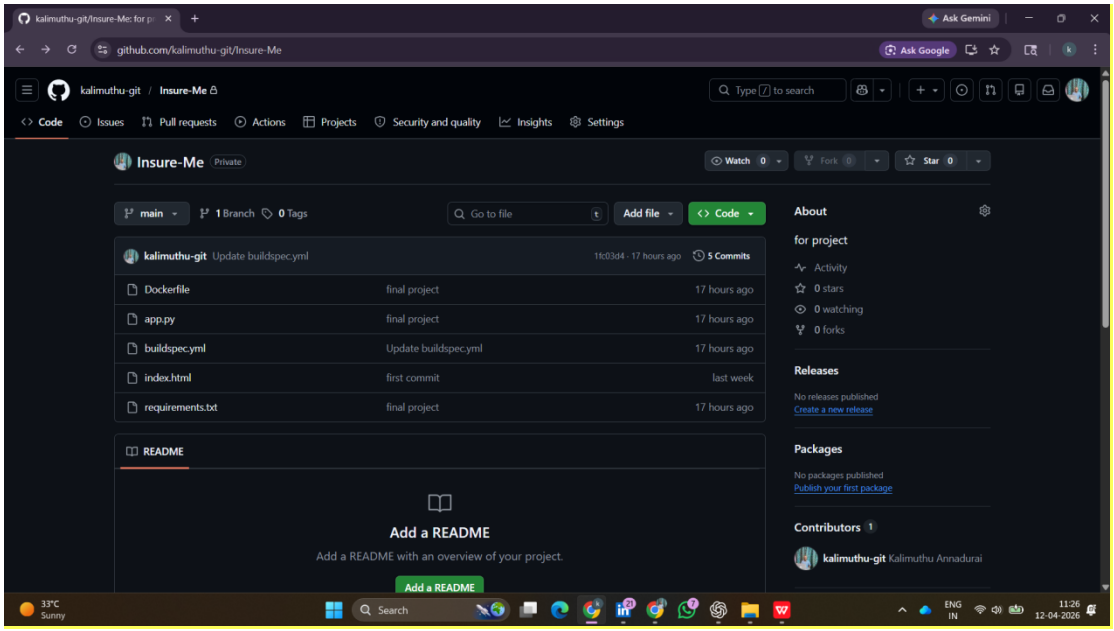
buildspec.yml file:

```
version: 0.2

phases:

pre_build:

commands:

- echo Logging in to Amazon ECR...
```

```
- aws --version

# Set variables

- REPOSITORY_URI=212045812865.dkr.ecr.us-east-1.amazonaws.com/project/insure-me

- IMAGE_TAG=latest

# Login to ECR

- aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin

212045812865.dkr.ecr.us-east-1.amazonaws.com

build:

commands:

- echo Build started on `date`

# Debug (optional but useful)

- echo "Files in directory:"

- ls -la

# Build Docker image

- docker build -t $REPOSITORY_URI:$IMAGE_TAG .

post_build:

commands:

- echo Pushing Docker image to ECR...

# Push image

- docker push $REPOSITORY_URI:$IMAGE_TAG

# Create imagedefinitions.json for ECS

- echo Creating imagedefinitions.json...

- printf '[{"name":"insure-me-container","imageUri":"%s"}]'$REPOSITORY_URI:$IMAGE_TAG

> imagedefinitions.json

artifacts:

files:

- imagedefinitions.json
```

6. Push Code to GitHub

git init
git add .
git commit -m "my first commit"
git push -m origin

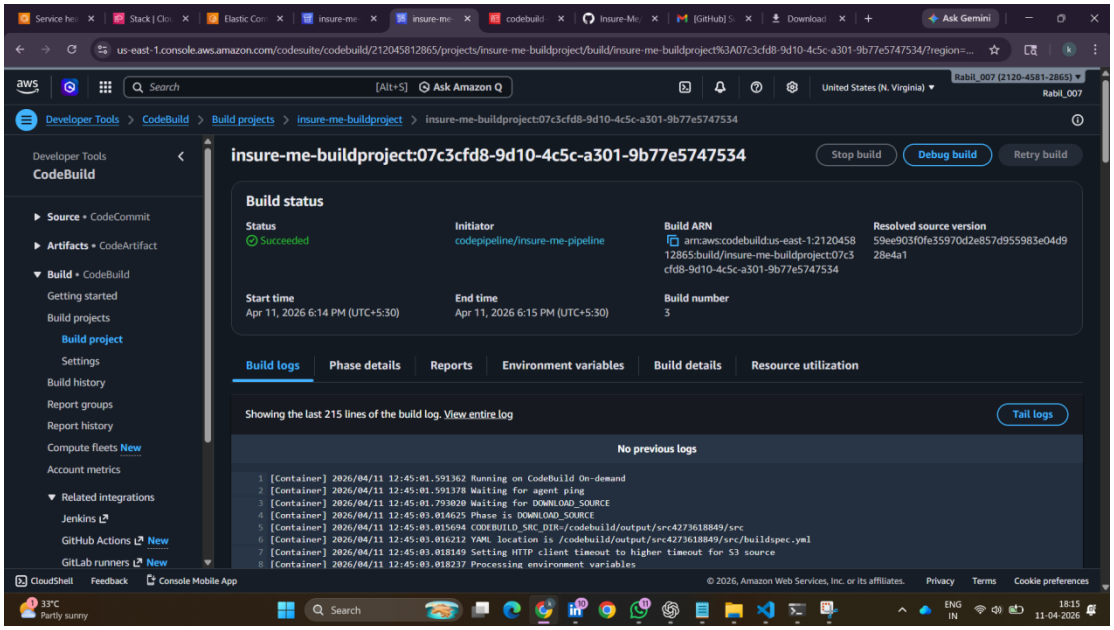


7. Create CodeBuild Project

Connect GitHub repo

Enable Docker

Use buildspec.yml



8. Create CodePipeline

Pipeline stages:

1.Source → GitHub

2.Build → CodeBuild

3.Deploy → ECS

9. Create ECS (Elastic Container Service) Cluster

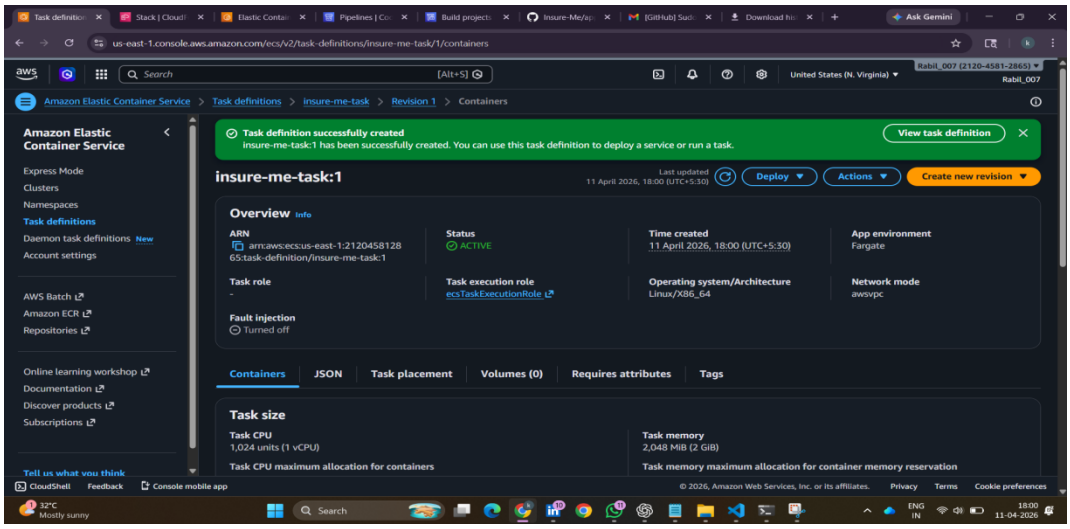
Launch type: Fargate

Create Task Definition:

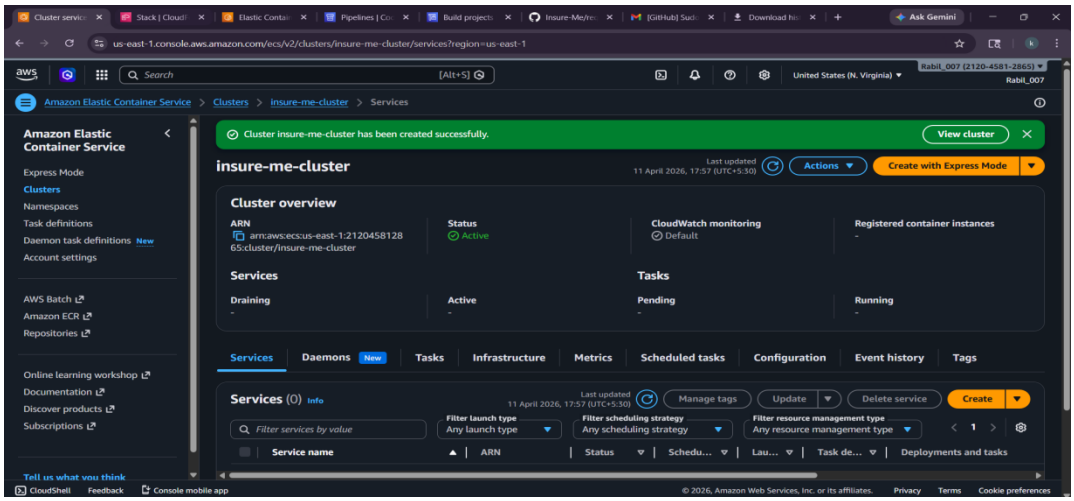
Container name: insure-me-container

Port: 5000

Create Task Definition created : Insure-me-task



Cluster Created : Insure-me-cluster

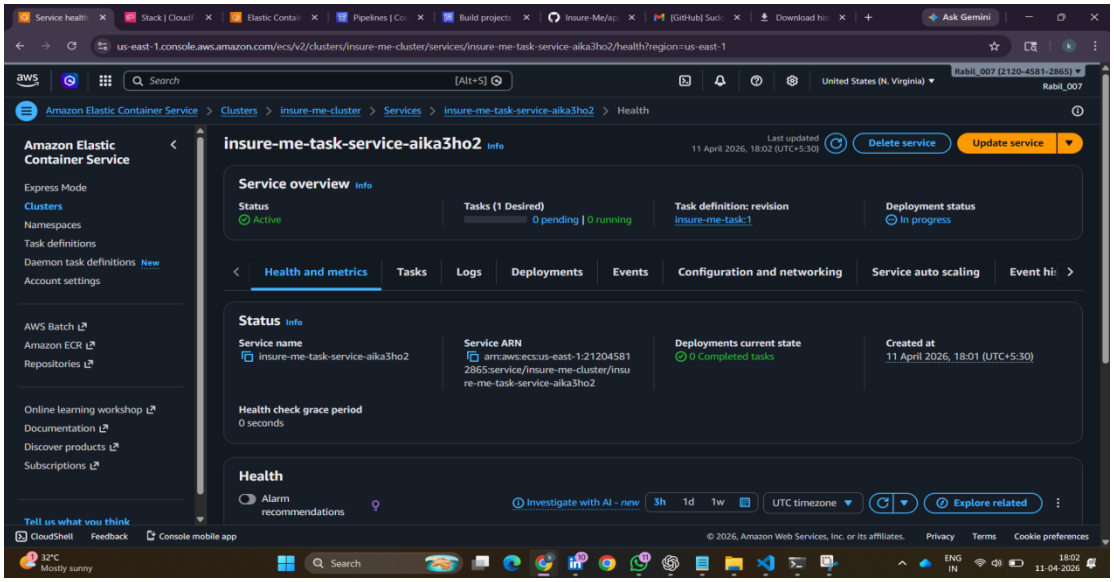


10. Deploy Application in ECS

Create Service

Attach Load Balancer

Service created : Insure-me-task-service-alka3ho2



11. Configure Load Balancer

Type: Application Load Balancer

Listener: HTTP (80)

Target: ECS container

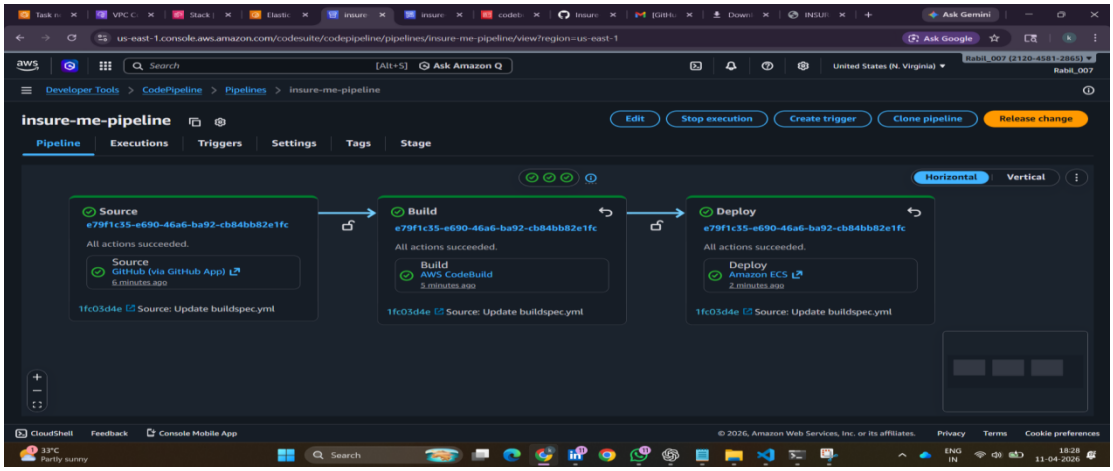
DNS Name : http://insure-me-alb-207979398.us-east.elb.amazonaws.com/

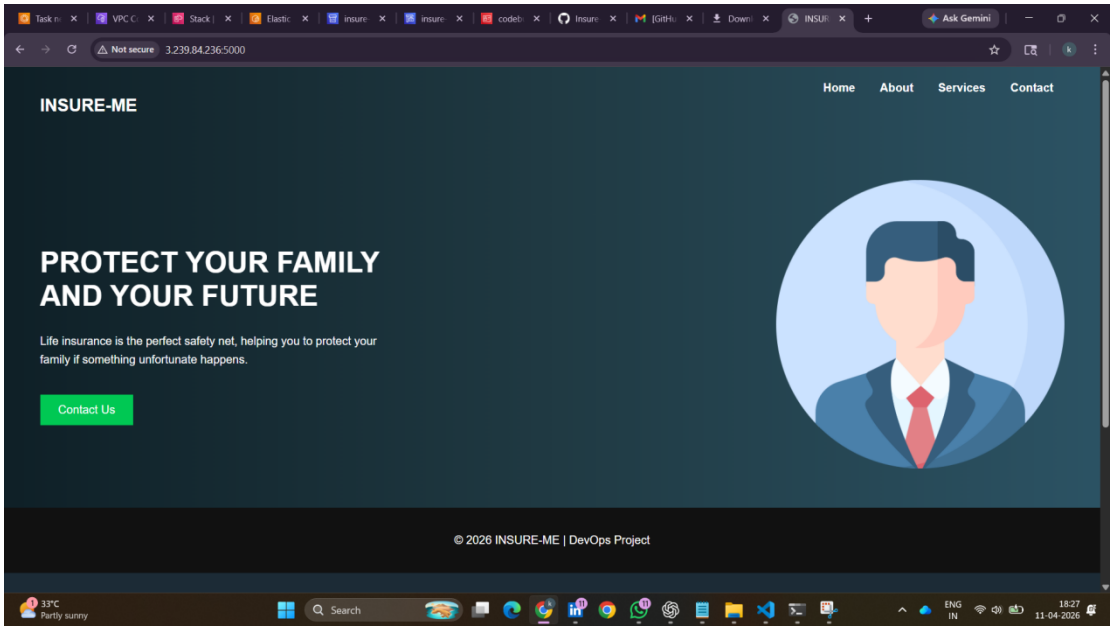
12.Final Output

Application deployed successfully

Accessible via Load Balancer URL

CI/CD pipeline working automatically





13.Key Features:

Automated CI/CD pipeline from GitHub to production

Containerized application using Docker

Scalable deployment using ECS Fargate

Image storage using Amazon ECR

Load balancing using Application Load Balancer

Real-time deployment on every code push

14.Conclusion:

The InsureMe project demonstrates a real-world DevOps pipeline using AWS services .

It provides hands-on experience in containerization, automation, and cloud deployment.